Chapter 6
Computer Arithmetic

2.)Sign-magnitude representation:-

→The most significant(left side) bit in the word as a sign bit.

Sign bit 0→ +Ve number

1→ -ve number

Eg.

+18 → 00010010

-18 → 10010010

Drawbacks:

Addition and subtraction require a consideration of both the sign of the numbers and their magnitude to carry out the required operation.

+0 = 0000 0000

-0= 1000 0000

This is inconvenient because it is slightly more difficult to test for 0, (which appears frequently on computers)

3.) Two's complement representation:-

Like sign magnitude, two's complement representation uses the most significant bit as a sign bit, making it easy to test whether it is +ve or –ve.

-7 = 1 1 1 1

Sign Magnitude

And its 2's complement is

=1001

5= 0 1 0 1

And its 2's complement is

=0101

Addition:-

-7 = 1 0 0 1	-4 = 1 1 0 0
+ 5 = 0 1 0 1	+ 4 = 0 1 0 0
-2 = 1 1 1 0	-0= 1 0 0 0 0

→The result may be larger that can be held in the word size being used. This condition is overflow.

→**Overflow rule**:- If two number are added, and they are both positive or negative than overflow occurs if and only if the result has opposite sign.

```
      5 = 0 1 0 1
      4 = 0 1 0 0
      ------------
        1 0 0 1
`    -7 = 1 0 0 1
     -6 = 1 0 1 0
      ------------
      1 0 0 1 1
```

Over flow

Subtraction:-

To subtract one number (subtrahend) from another number (minuend), take the 2's complement (negative) of the subtrahend and add it to the minuend.

M=2 s= 7

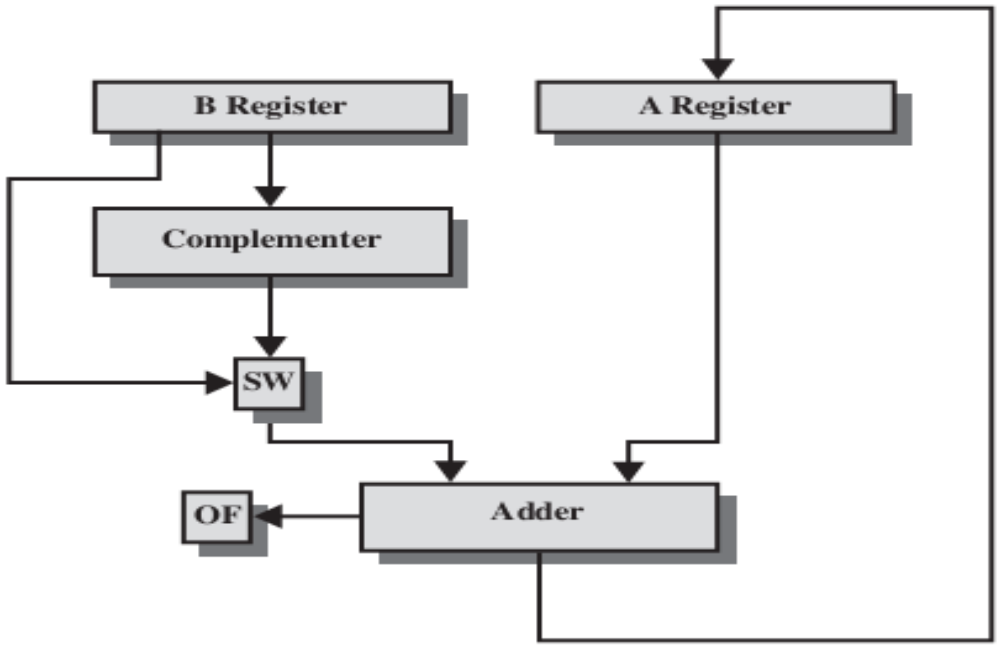
-7 = 2's complement of 0111

```
   =  1000
   +     1
   --------
     1001
```

Therefore, 2-7 = 2+ (-7)

```
   0010
   1001
   ----
   1011
```

Block Diagram of H/W Addition Subtraction:



OF = overflow bit
SW = Switch (select addition or subtraction)

Q. Explain the algorithm for signed magnitude number addition/subtraction with suitable examples.

Ans.:-

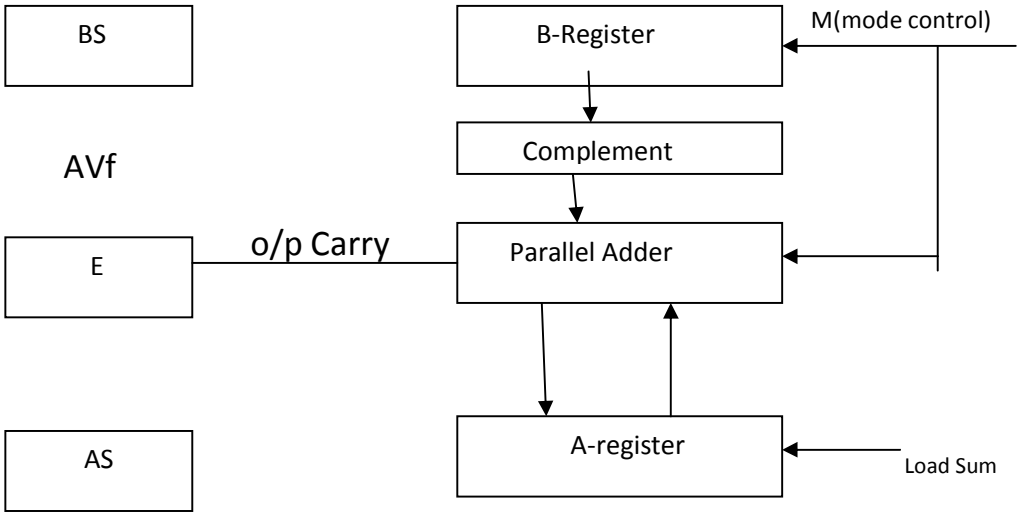


Fig: Hardware for signed magnitude addition & subtraction.

→Above block diagram for addition and subtraction consist of register A&B and sign flip-flop AS & BS subtraction is done by adding A to the 2's complement of B.

→The o/p carry is transfer flip-flop E where it can be checked to determine the relative magnitude of two magnitude.

→The add overflow flip-flop(AOF) holds the overflow when A & B are added.

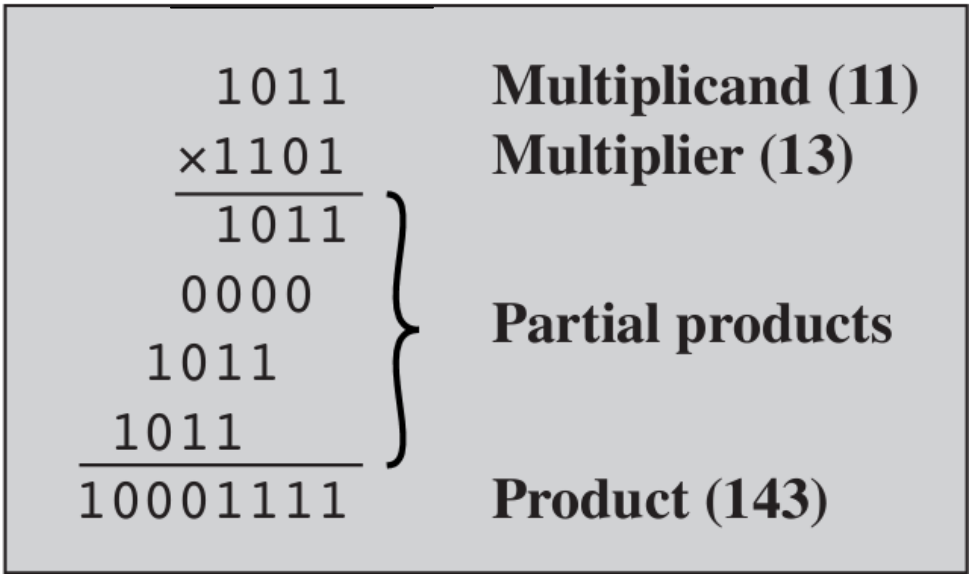
→The A register provides other micro operation that may be needed when we specify sequence of steps.

→The addition of A+B is done through the parallel adder.

→The complement provides the o/p of B or the complement of B depending on the state of mode control M. The M signal also applied to the i/p carry of the adder when M=0, the value of B is transfer to the adder, the i/p carry is zero & the o/p of the adder is equal to sum A+B.

→When M=1, the one's complement of B is applied to the adder, the i/p carry is I & O/P = A+B+I which is equal to A+2's complement of B. Again which is equivalent to the subtraction A-B.

Multiplication:

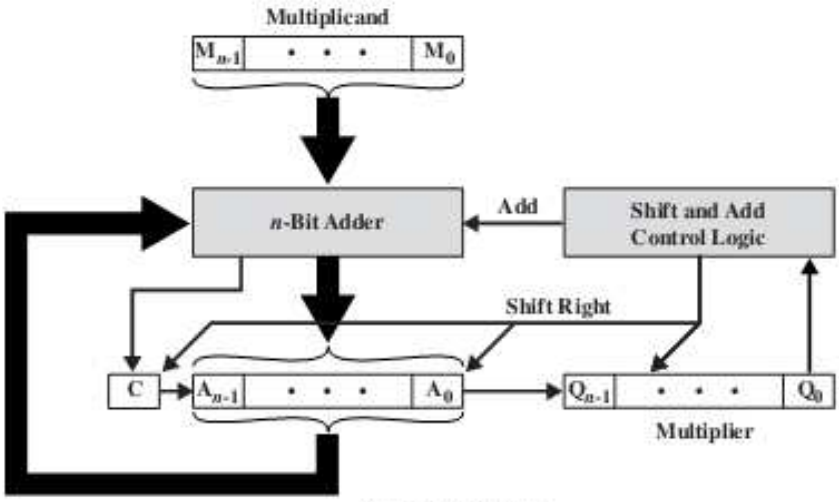


Example1:- Multiplication Of Unsigned Binary Integers

$(1011)_2 * (1101)_2 = (10001111)_2$

(11*13) = (143)

Block Diagram of multiplication:-



(a) Block Diagram

C	A	Q	M		
0	0000	1101	1011	Initial Values	
0	1011	1101	1011	Add	First Cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	Shift	Second Cycle
0	1101	1111	1011	Add	Third Cycle
0	0110	1111	1011	Shift	
1	0001	1111	1011	Add	Fourth Cycle
0	1000	1111	1011	Shift	

(b) Above Example 1

Fig. Hardware Implementation of Unsigned Binary Multiplication

Another Example of Multiplication:-

1010*1100

	C	A	Q	M	
	0	0000	1010	1100	
Since,Q0=0	0	0000	0101	1100 → 1st Cycle	
Since,Q0=1		C,A ← A+M			
	0	1100	0101	1100—add	2nd
	0	0110	0010	1100—shift	
Cycle					
Since,Q0=0					
	0	0011	0001	1100—shift	3rd
Cycle					
Since,Q0=1		C,A ← A+M			
	0	1111	0001	1100—add	4th
Cycle					
	0	0111	1000	1100—shift	

Answer = 0111 1000

Flowchart of Multiplication:-

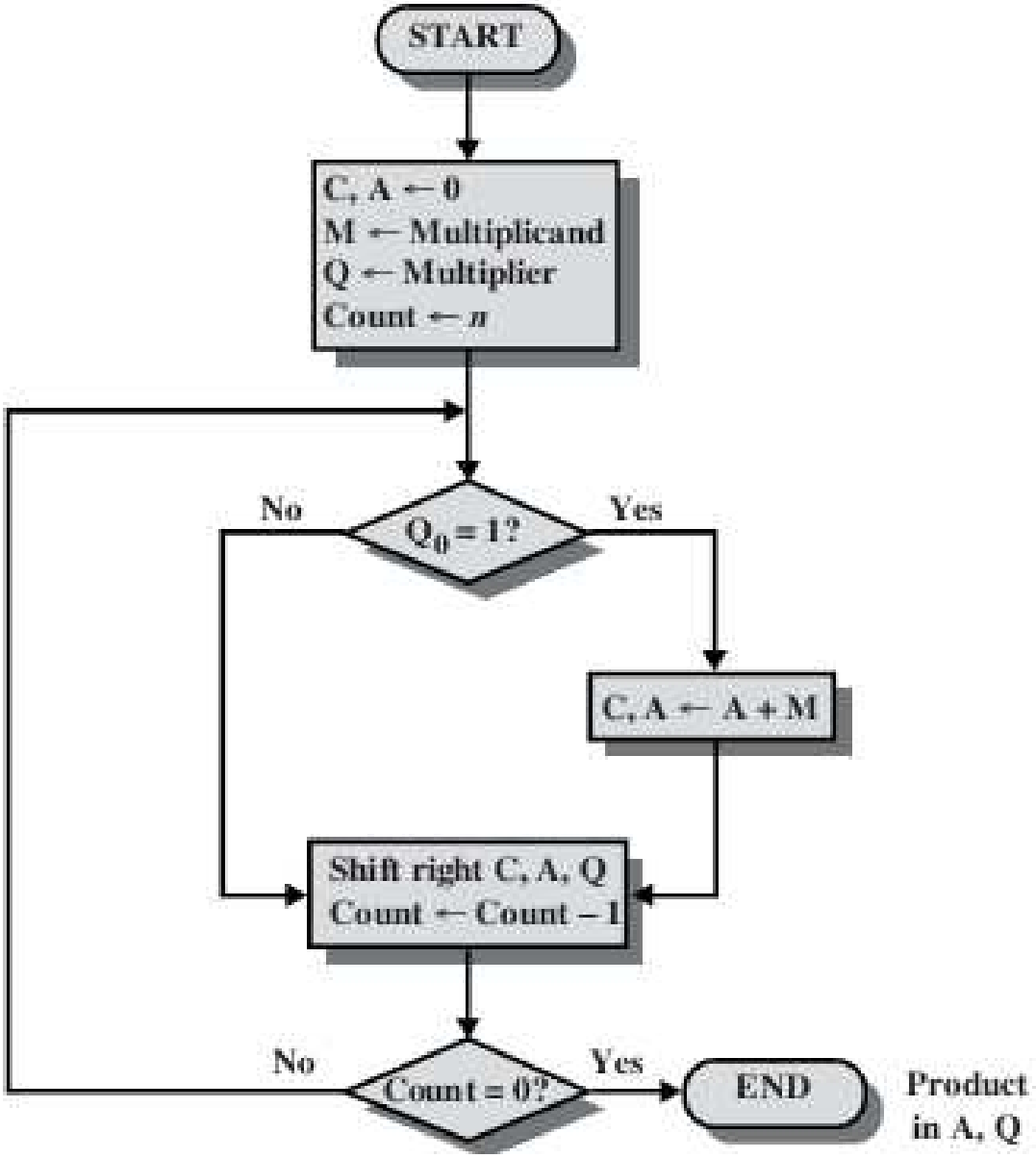


Fig. Flow chart of unsigned binary multiplication

#. Two's complement Multiplication Booth's Algorithm:-

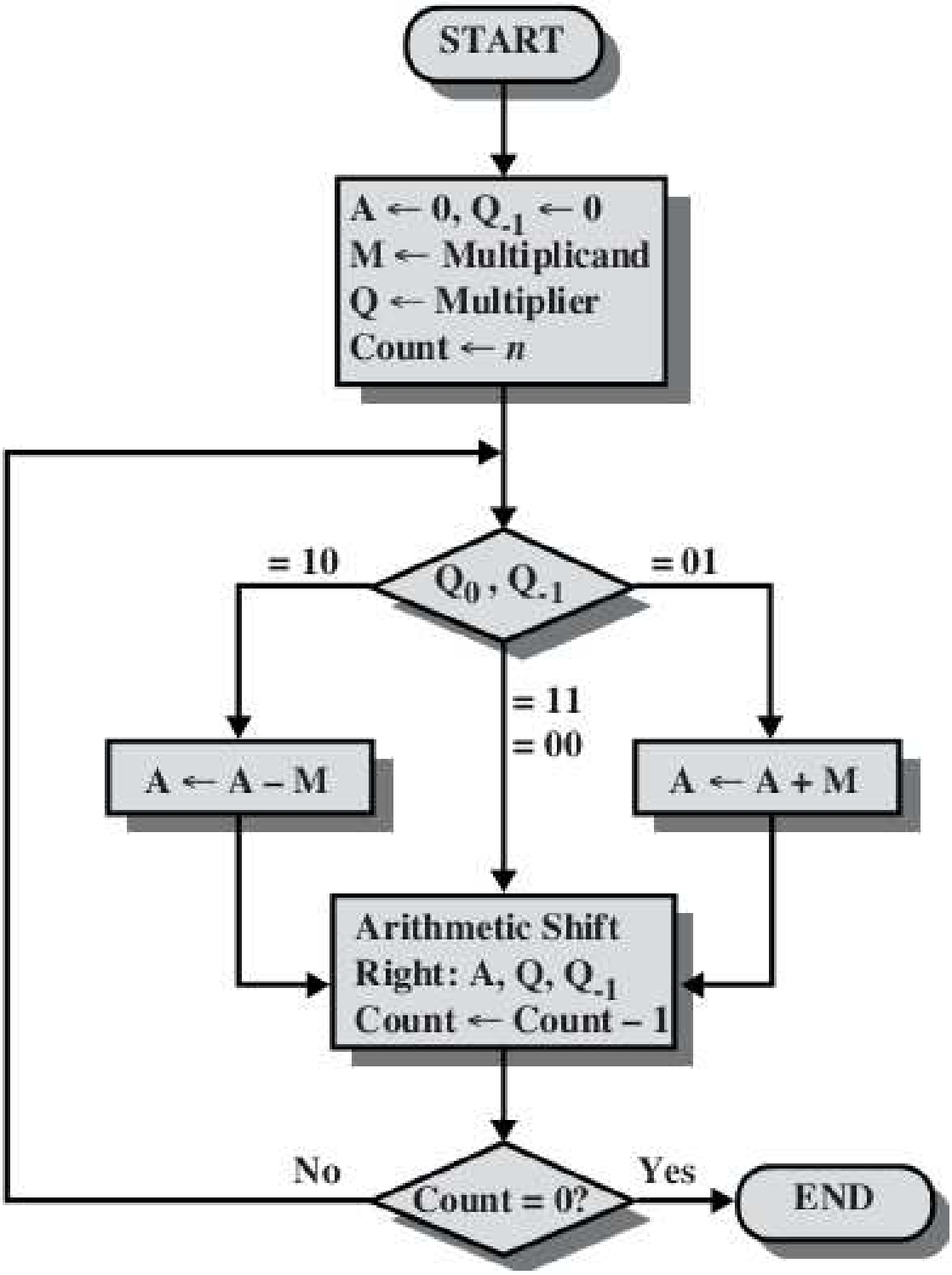


Fig. Booth's Algorithm for twos Complement Multiplication

Example:

Booth's Algorithm (7*3)

A	Q	Q_{-1}	M		
0000	0011	0	0111	Initial Values	
1001	0011	0	0111	A ← A - M	First Cycle
1100	1001	1	0111	Shift	
1110	0100	1	0111	Shift	Second Cycle
0101	0100	1	0111	A ← A + M	Third Cycle
0010	1010	0	0111	Shift	
0001	0101	0	0111	Shift	Fourth Cycle

→Since, QQ_{-1} =11

→Since, QQ_{-1} =01

→Since, QQ_{-1}=00

AQ= 0001 0101

Answer

Another Example:

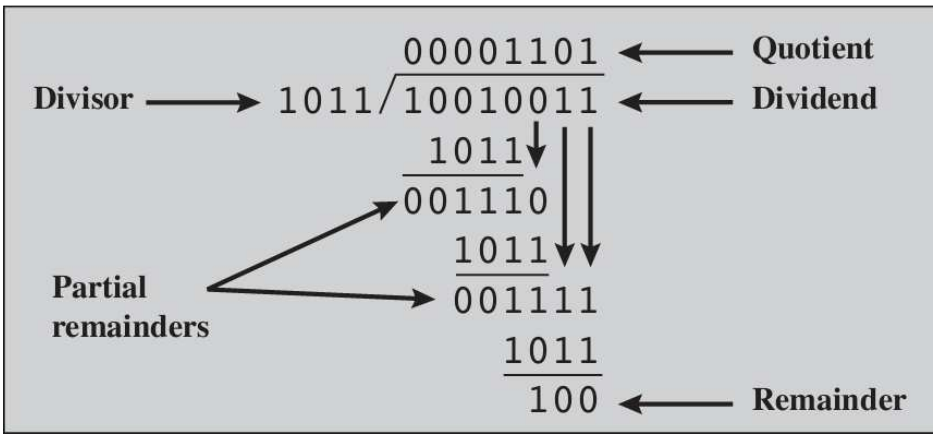
-7*3

	A	Q	Q_{-1}	M	
	000	0011	0	1001	
QQ_{-1}=10	A← A-M				
	0111	0011	0	1001	A←A-M
	0011	1001	1	1001	Shift
Q$Q_{-1} = 11$					
	0001	1100	1	1001	Shift
QQ_{-1}=01	A←A+M				
	1010	1100	1	1001	
	1101	0110	0	1001	
QQ_{-1}=00					
	1110	1011	0	1001	

Note:

In either of the case, the right shift is such that the leftmost bit of A namely A_{n-1}, not only is shifted into A_{n-2} but also remains in A_{n-1}. This is required to preserve the sign of the number in A&Q. It is known as arithmetic shift it preserves sign bit.

Division: Division of unsigned binary division.



Flowchart of unsigned Binary division.

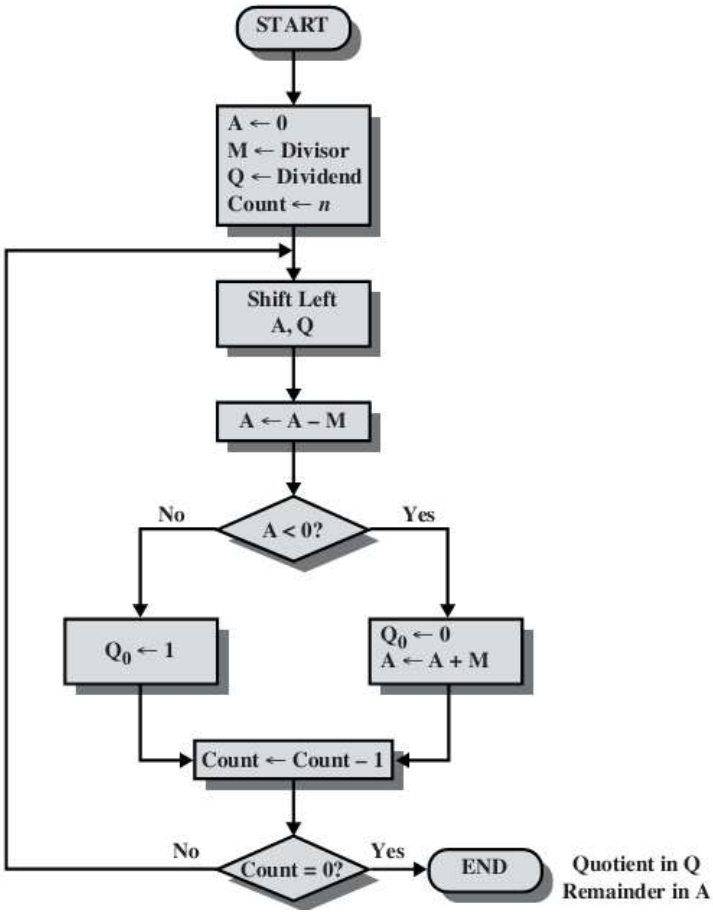


Fig. FLOWCHART UNSIGNED BINARY DIVISION

Two's complement division Algorithm:

1. M←Divisor

   A, Q←Dividend (expressed as 2n bits)

   For eg.

   0111 becomes 0000 0111

   1001 becomes 1111 1001

2. Shift A, Q left 1 Bit position.

3. If M & A has same sign perform

   A←A-M,

   Otherwise, A←A+M

4. The step 3 is successful if the sign of A is same before and after the operation.

   a. If Successful or A=0, then set Q=1

   b. If unsuccessful and A≠0, set Q←0 & restore previous value of A.

5. Repeat step 2 through 4 for 'n' times.

6. The reminder is in A. If the sign of divisor & dividend are same quotient is in Q. Otherwise quotient is 2's complement of Q.

$$D=Q*V+R$$

Where, D=Dividend V= Divisor

Q= Quotient R=Reminder.

Examples of twos complement Division

A	Q	M = 0011
0000	0111	Initial value
0000	1110	shift
1101		subtract
0000	1110	restore
0001	1100	shift
1110		subtract
0001	1100	restore
0011	1000	shift
0000		subtract
0000	1001	set $Q_0 = 1$
0001	0010	shift
1110		subtract
0001	0010	restore

(a) (7)/(3)

A	Q	M = 1101
0000	0111	Initial value
0000	1110	shift
1101		add
0000	1110	restore
0001	1100	shift
1110		add
0001	1100	restore
0011	1000	shift
0000		add
0000	1001	set $Q_0 = 1$
0001	0010	shift
1110		add
0001	0010	restore

(b) (7)/(–3)

A	Q	M = 0011
1111	1001	Initial value
1111	0010	shift
0010		add
1111	0010	restore
1110	0100	shift
0001		add
1110	0100	restore
1100	1000	shift
1111		add
1111	1001	set $Q_0 = 1$
1111	0010	shift
0010		add
1111	0010	restore

(c) (–7)/(3)

A	Q	M = 1101
1111	1001	Initial value
1111	0010	shift
0010		subtract
1111	0010	restore
1110	0100	shift
0001		subtract
1110	0100	restore
1100	1000	shift
1111		subtract
1111	1001	set $Q_0 = 1$
1111	0010	shift
0010		subtract
1111	0010	restore

(d) (–7)/(–3)

4.)Floating Points:

Floating point numbers: Two Uses:

1. Real no. with non zero fractions

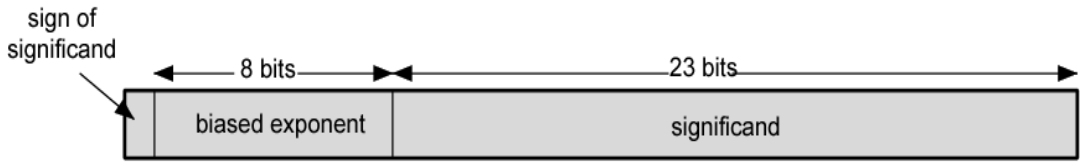
3.1416 , $6.62*10^{-23}$

2. Really high number

$3*10^{8}$, $6.02*10^{23}$ ← Exponent

Mantissa Radix

Floating point representation.



(a) Format

→+/- .significand x $2^{exponent}$

→Misnomer

→Point is actually fixed between sign bit and body of mantissa

→Exponent indicates place value (point position)

examples

$1.1010001 \times 2^{10100} = 0\ 10010011\ 10100010000000000000000 = 1.6328125 \times 2^{20}$

$-1.1010001 \times 2^{10100} = 1\ 10010011\ 10100010000000000000000 = -1.6328125 \times 2^{20}$

$1.1010001 \times 2^{-10100} = 0\ 01101011\ 10100010000000000000000 = 1.6328125 \times 2^{-20}$

$-1.1010001 \times 2^{-10100} = 1\ 01101011\ 10100010000000000000000 = -1.6328125 \times 2^{-20}$

#Addition and subtraction:

Four phases of the algorithm for addition and subtraction.

1. Check for zeros.

→Addition ~ subtraction except sign change.

→If either operand is 0, the other is reported as result.

2. Significand Alignment

→Manipulate numbers so that two exponent are equel.

Eg:- $(123 * 10^0) + (456 * 10^{-2})$

$= (123 * 10^0) + (4.56 * 10^0)$

$=127.56 * 10^0$

→Alignment may be achieved by shifting either smaller number to right (increasing its exponent) or shifting the larger number to left.

3. Addition

→Significands added taking account their sign bit also.

→Possibility of significand overflow by 1 digit. If so, The Significand of result is shifted right and exponent is incremented

4. Normalization.

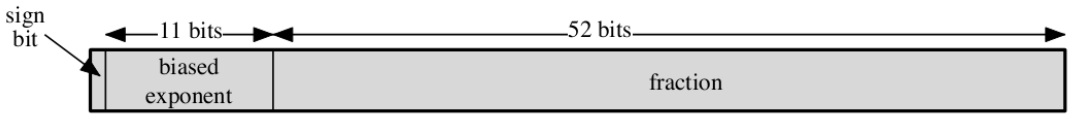
→ Significand digits are shifted left until MSD is non-zero.

→Each shift causes a decrement of the exponent and thus could cause exponent underflow.

→Finally the result must be rounded off and then reported.

IEEE 754 standard format for floating point.

(a) Single format



(b) Double format

Example : Represent $(47.625)_{10}$ *in Single Precision Floating Point (SPFP) format?*

Step:1

47=101111

0.625*2=1.250 →1

0.25*2=0.5 →0

0.5*2=1 →1

$(47.625)_{10}$= 101111.101

Step:2

1.01111101*2^5 (Normalization form)

=1.01111101*2^5

Step:3

Real Exp= 5+127 ← Biased

=132

Binary of 132

128	64	32	16	8	4	2	1
1	0	0	0	0	1	0	0

132=1000100

Sign=+ve

0	10000100	01111101000000000000000

8 bit 23 bit

5.) Flowchart of floating point addition/subtraction

The floating point addition/subtraction are more complex than multiplication and division.

This is because of the need for alignment. There are four basic phases of the algorithm for addition and subtraction. These phases are clearly shown in below figure (flowchart). Namely

1. Check for zeros
2. Align for significands
3. Add or subtract the Significands
4. Normalization the result

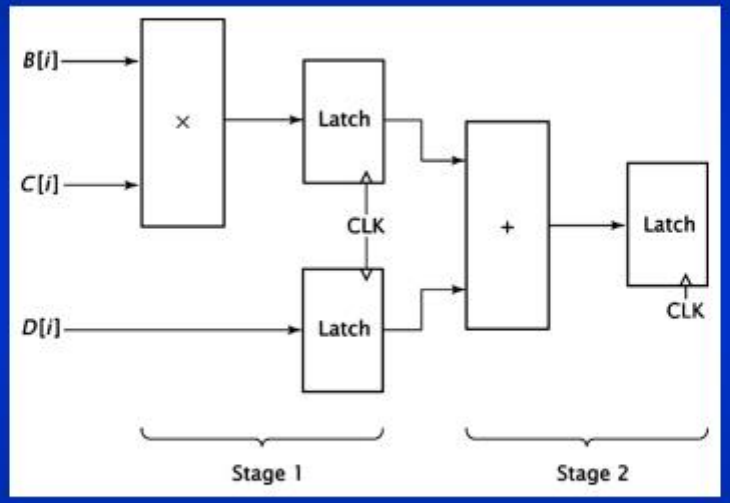
<u>The first phase</u> is zero check. Because addition and subtraction are identical except for a sign change, the process begins by the subtratend if it is a subtract operation. Next, if either operand is 0, the other reported as the result.

<u>The second phase</u> is significand alignment. The numbers are manipulated so that the two exponents are equal. Alignment can be achieved by shifting either the smaller number to the right (increasing its exponent) or shifting

Specialized Arithmetic hardware

1. **PIPELINING**

FOR i = 1 TO 100 DO $\{A[i] \leftarrow (B[i] \cdot C[i]) + D[i]\}$



$$S_{100} = \frac{nT_1}{(k+n-1)T_k} = \frac{100 \cdot 20\text{ ns}}{(2+100-1) \cdot 10\text{ ns}} = 1.98$$

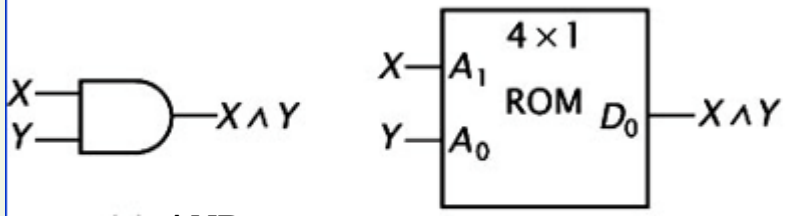
T_1 = time taken by non-pipelined unit to calculate 1 result. Requires $n.T_1$ time to calculate n results.
Pipelined unit requires k time units, each of duration T_k , to move the first piece of data through the pipeline.
Because, additional data enters the pipeline during every cycle, it will output the remaining n-1 results during next n-1 cycles.
=> thus, the pipelined unit requires (k + n – 1) cycles, each of T_k time, to calculate same results.
The speedup can be calculated by the formula above.

Contd…..

- Data enters into a stage of the pipeline, which performs some arithmetic operations on those data.
- The results are then passed to next stage which performs its operation and so on until the final computation ha been performed.
- Each stage performs only its specific function, no need to be capable of performing the task of any other stage.
- A pipeline does not speed up an individual computation. It improves the performance by overlapping computations where each stage can operate on different data simultaneously.
- The net effect is that output appears more quickly than in non-pipeline arithmetic unit, thereby increasing the throughput.

2. Lookup Tables:

- Any combinational circuit can be implemented by ROM is a lookup table.
- Inputs to the combinational circuit serve as the address input of the ROM.
- Data outputs of the ROM corresponds to the output of the combinational circuit.
- ROM is programmed with data such that the correct values are output for any possible input values.
- Consider a 4 x 1 ROM programmed to mimic a two input AND gate.



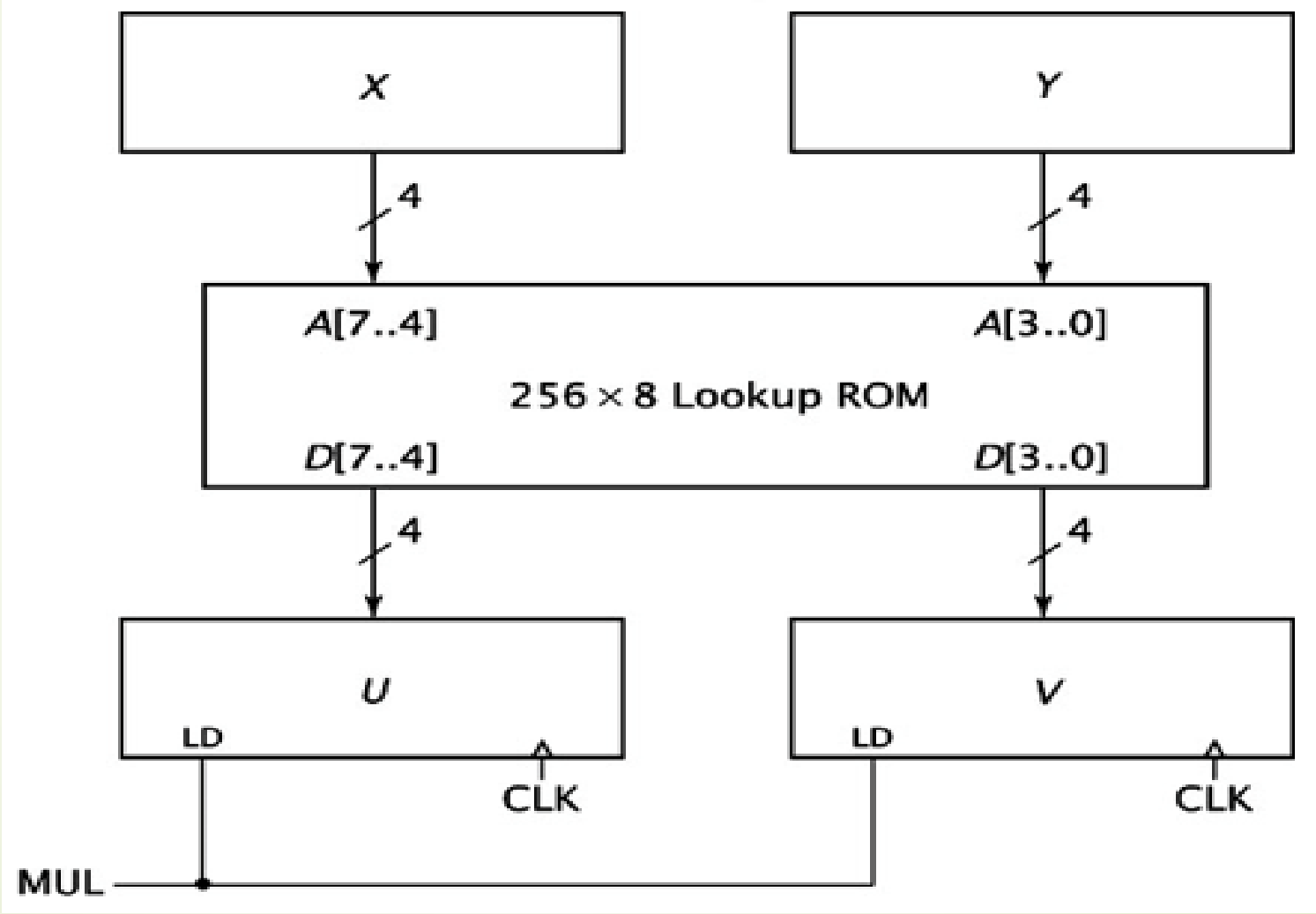
(a) AND gate

Address		Data
0	0	0
0	1	0
1	0	0
1	1	1

(b) Lookup ROM equivalent

- By programming ROM with data shown, it outputs the same values as the AND gate for all possible values of X and Y.

Contd..(a multiplier implemented using a lookup ROM)

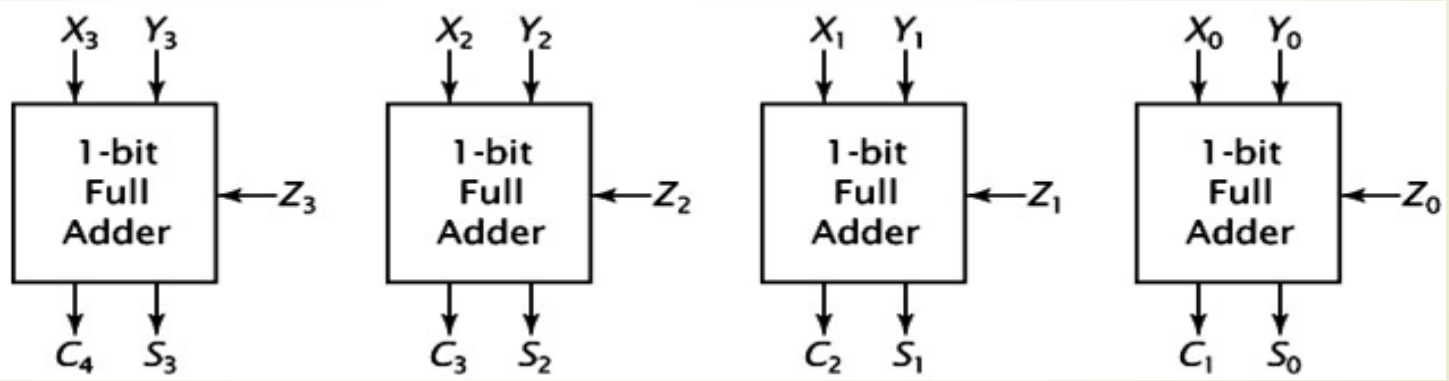


Contd…

- Registers X and Y supply the address inputs to lookup ROM.
- It’s outputs are the product of X and Y, and are routed to registers U and V.
- Each of 256 locations must contain the 8-bit product of X and Y.
- For example, location 1011 1101 contains the data 1000 1111, 143, the product of 1011(11) and 1101(13).
- Advantages:
  => the hardware may be less complex than that of the original shift-add implementation.
  => can multiply numbers more quickly than the shift-add hardware.
  => However, size of the lookup ROM grows rapidly as the size of the operands increases.

3. Wallace Tree

- Combinational circuit used to multiply two numbers.
- Instead of performing additions using standard parallel adder, Wallace tree uses carry save adder and only one parallel adder.
- Carry save adder can add 3 values simultaneously. However, it does not output a single result, but provides both sum and a set of carry bits as output.



- To form the final sum, S and C must be added together because carry bits do not propagate through the adder.

- To use a carry save adder to perform multiplication, we first calculate the partial product of the multiplication and input them to carry save adder.

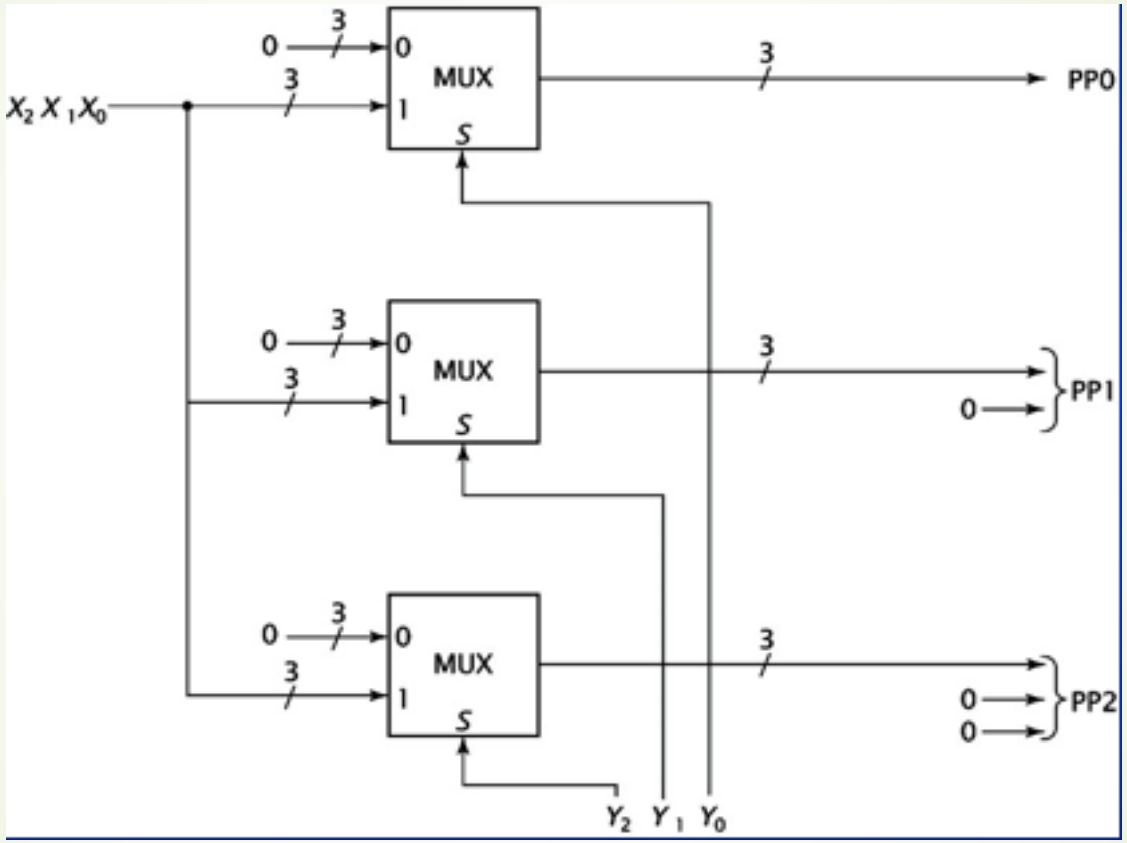
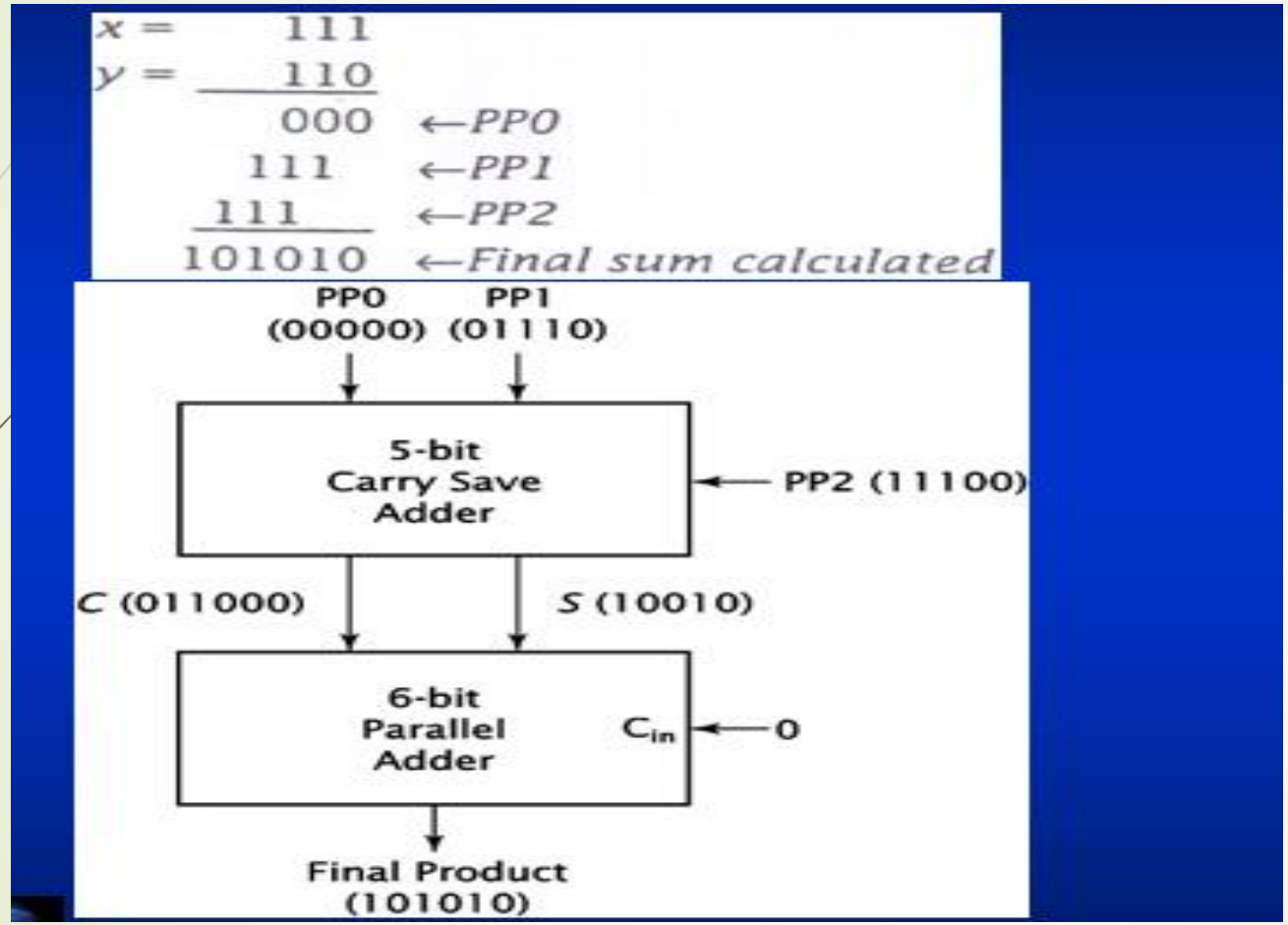


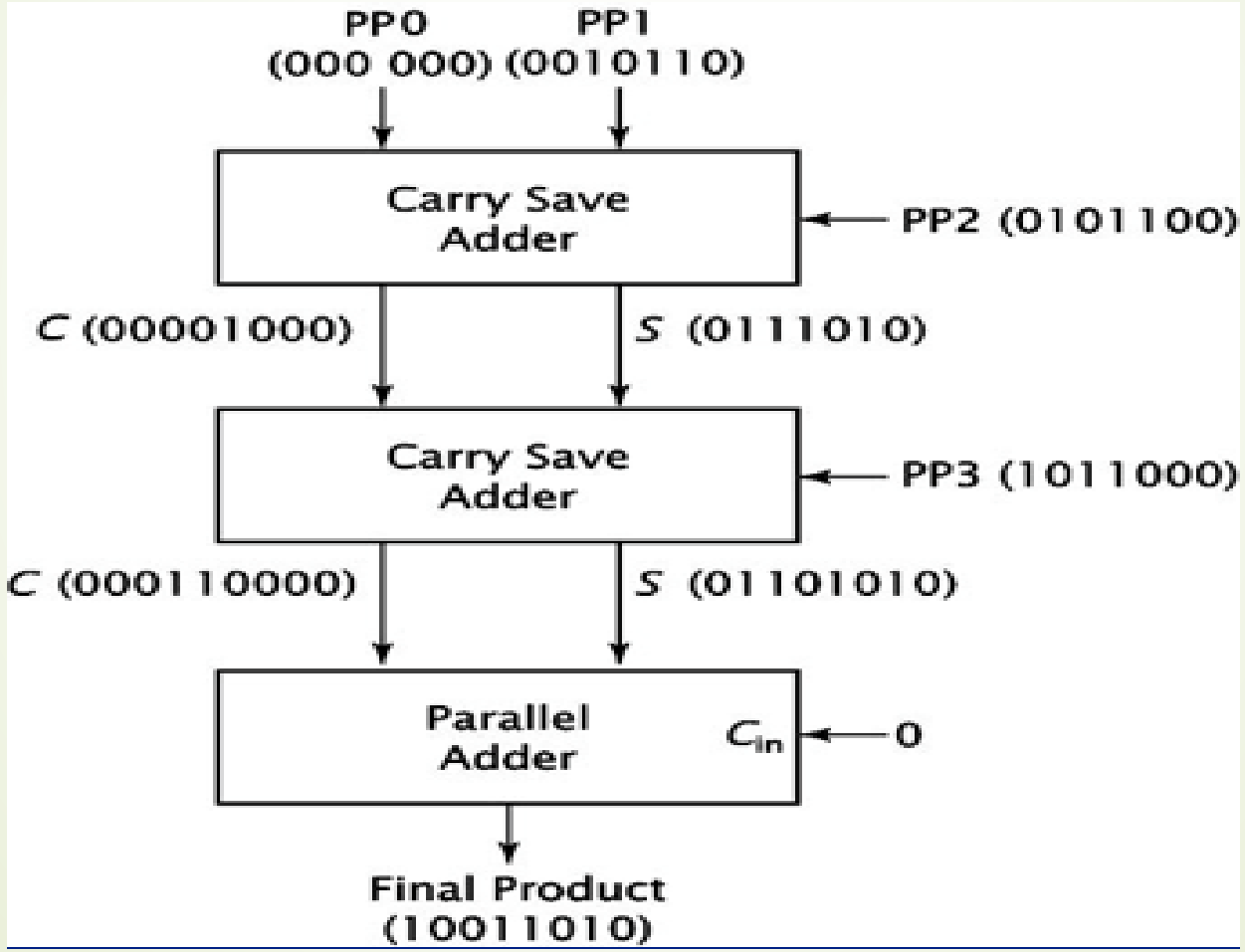
Fig: Generating partial products for multiplication using Wallace tree.

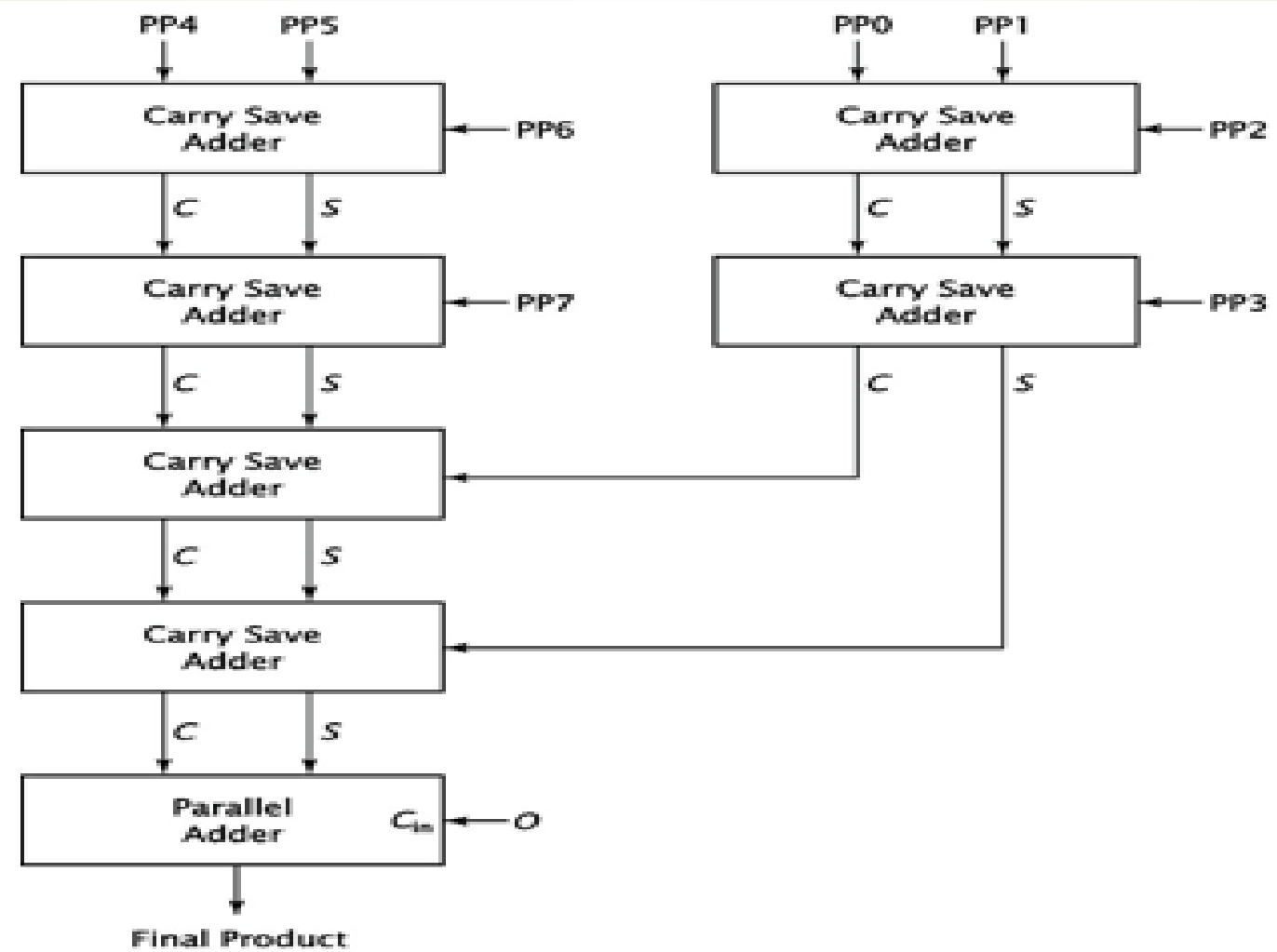
A 3 x 3 multiplier constructed using carry save adder.



Contd..

A 4 x 4 Wallace tree multiplier.



Contd…

An 8 x 8 Wallace tree multiplier.